Ex. No. 7	**Data Persistence using Google Cloud Firestore**
Date of Exercise	
Github Code Link (Keep as public)	
Video Link (Google Drive Link)	

Aim

Develop a Flutter application that stores and retrieves employee or student information using Google Cloud Firestore, ensuring data persistence even after the application is closed and reopened.

Description

Cloud Firestore is a NoSQL document database hosted on Google Cloud; unlike SQLite it persists data on a remote server rather than on the device, so records survive not only app restarts but also reinstallation, and can be shared across devices and users. The `cloud_firestore` and `firebase_core` packages connect a Flutter app to a specific Firebase project through a FirebaseOptions object (apiKey, appId, projectId, etc.) that is normally generated by running `flutterfire configure` against a real project.

This development environment has no Firebase project or credentials of its own, so lib/firebase_options.dart ships with clearly-labelled placeholder values ('REPLACE_WITH_YOUR_API_KEY', ...) and a comment explaining that `flutterfire configure` must be run to replace it before the screen can reach a real backend. main.dart checks for that placeholder apiKey before calling Firebase.initializeApp; when it is detected, the app still renders the full UI but sets a firebaseReady flag to false instead of attempting a network call that would otherwise hang. PeopleRecordsScreen uses that flag to disable the 'Save to Firestore' button, show an orange banner telling the user to connect a real project, and display 'No data — connect Firestore to see records here' instead of a spinner or a crash. Once a real project is configured, firebaseReady becomes true, the banner disappears, and the same PeopleRepository.addPerson / watchPeople methods (add() and a snapshots() StreamBuilder on the 'people' collection) persist and live-stream employee/student records to and from Cloud Firestore with no further code changes.

Program

lib/models/person_record.dart

```
class PersonRecord {
  final String? id;
  final String name;
```

```
  final String rollOrEmpId;
  final String department;

  const PersonRecord({
    this.id,
    required this.name,
    required this.rollOrEmpId,
    required this.department,
  });

  Map<String, Object?> toMap() => {
        'name': name,
        'rollOrEmpId': rollOrEmpId,
        'department': department,
      };

  factory PersonRecord.fromMap(String id, Map<String, Object?> map) =>
PersonRecord(
        id: id,
        name: map['name'] as String? ?? '',
        rollOrEmpId: map['rollOrEmpId'] as String? ?? '',
        department: map['department'] as String? ?? '',
      );
}
```

lib/services/people_repository.dart

```
import 'package:cloud_firestore/cloud_firestore.dart';

import '../models/person_record.dart';

/// Thin wrapper around the `people` Firestore collection used to store
/// employee / student records. Requires a real Firebase project — see
/// lib/firebase_options.dart.
class PeopleRepository {
  final CollectionReference<Map<String, Object?>> _collection =
      FirebaseFirestore.instance.collection('people');

  Future<void> addPerson(PersonRecord person) =>
_collection.add(person.toMap());

  Stream<List<PersonRecord>> watchPeople() {
    return _collection.orderBy('name').snapshots().map(
          (snapshot) =>
              snapshot.docs.map((doc) => PersonRecord.fromMap(doc.id,
doc.data())).toList(),
        );
  }
}
```

lib/screens/people_records_screen.dart

```
import 'package:flutter/material.dart';

import '../models/person_record.dart';
import '../services/people_repository.dart';

class PeopleRecordsScreen extends StatefulWidget {
  final bool firebaseReady;

  const PeopleRecordsScreen({super.key, required this.firebaseReady});

  @override
  State<PeopleRecordsScreen> createState() => _PeopleRecordsScreenState();
}

class _PeopleRecordsScreenState extends State<PeopleRecordsScreen> {
  final _formKey = GlobalKey<FormState>();
  final _nameController = TextEditingController();
  final _idController = TextEditingController();
```

```
  final _deptController = TextEditingController();

  PeopleRepository? _repository;

  @override
  void initState() {
    super.initState();
    if (widget.firebaseReady) {
      _repository = PeopleRepository();
    }
  }

  @override
  void dispose() {
    _nameController.dispose();
    _idController.dispose();
    _deptController.dispose();
    super.dispose();
  }

  String? _required(String? v) => (v == null || v.trim().isEmpty) ? 'Required'
: null;

  Future<void> _save() async {
    if (!_formKey.currentState!.validate() || _repository == null) return;
    await _repository!.addPerson(PersonRecord(
      name: _nameController.text.trim(),
      rollOrEmpId: _idController.text.trim(),
      department: _deptController.text.trim(),
    ));
    _nameController.clear();
    _idController.clear();
    _deptController.clear();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Employee / Student Records
(Firestore)')),
      body: Column(
        children: [
          if (!widget.firebaseReady)
            Container(
              width: double.infinity,
              color: Colors.orange.shade100,
              padding: const EdgeInsets.all(12),
              child: const Text(
                'Firebase not configured — run `flutterfire configure` and
connect a '
                'real Firebase project to use this screen.',
                style: TextStyle(fontWeight: FontWeight.w600),
              ),
            ),
          Padding(
            padding: const EdgeInsets.all(16),
            child: Form(
              key: _formKey,
              child: Column(
                children: [
                  TextFormField(
                    controller: _nameController,
                    decoration: const InputDecoration(labelText: 'Name',
border: OutlineInputBorder()),
                    validator: _required,
                  ),
                  const SizedBox(height: 10),
                  TextFormField(
                    controller: _idController,
                    decoration:
                        const InputDecoration(labelText: 'Roll No. / Employee
ID', border: OutlineInputBorder()),
```

```
                    validator: _required,
                  ),
                  const SizedBox(height: 10),
                  TextFormField(
                    controller: _deptController,
                    decoration: const InputDecoration(labelText: 'Department',
border: OutlineInputBorder()),
                    validator: _required,
                  ),
                  const SizedBox(height: 12),
                  SizedBox(
                    width: double.infinity,
                    child: ElevatedButton.icon(
                      onPressed: widget.firebaseReady ? _save : null,
                      icon: const Icon(Icons.cloud_upload_outlined),
                      label: const Text('Save to Firestore'),
                    ),
                  ),
                ],
              ),
            ),
          ),
          const Divider(height: 1),
          Expanded(child: _buildList()),
        ],
      ),
    );
  }

  Widget _buildList() {
    if (_repository == null) {
      return const Center(child: Text('No data — connect Firestore to see
records here.'));
    }
    return StreamBuilder<List<PersonRecord>>(
      stream: _repository!.watchPeople(),
      builder: (context, snapshot) {
        if (snapshot.hasError) {
          return const Center(child: Text('No data — connect Firestore to see
records here.'));
        }
        if (!snapshot.hasData) {
          return const Center(child: CircularProgressIndicator());
        }
        final people = snapshot.data!;
        if (people.isEmpty) {
          return const Center(child: Text('No records yet.'));
        }
        return ListView.builder(
          padding: const EdgeInsets.symmetric(vertical: 8),
          itemCount: people.length,
          itemBuilder: (context, i) {
            final p = people[i];
            return Card(
              margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
              child: ListTile(
                title: Text(p.name, style: const TextStyle(fontWeight:
FontWeight.bold)),
                subtitle: Text('${p.department} • ${p.rollOrEmpId}'),
              ),
            );
          },
        );
      },
    );
  }
}
```

lib/firebase_options.dart (placeholder — run flutterfire configure)

```
// Placeholder Firebase configuration.
//
// This project has not been connected to a real Firebase project yet.
// Before running this experiment against live Firestore data, install the
// FlutterFire CLI and run:
//
//   dart pub global activate flutterfire_cli
//   flutterfire configure
//
// That command will overwrite this file with the real options generated
// for your own Firebase project (web, Android, iOS, etc.).

import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;
import 'package:flutter/foundation.dart' show kIsWeb, TargetPlatform,
defaultTargetPlatform;

class DefaultFirebaseOptions {
  static FirebaseOptions get currentPlatform {
    if (kIsWeb) return web;
    switch (defaultTargetPlatform) {
      case TargetPlatform.android:
        return android;
      case TargetPlatform.iOS:
        return ios;
      default:
        return web;
    }
  }

  static const FirebaseOptions web = FirebaseOptions(
    apiKey: 'REPLACE_WITH_YOUR_API_KEY',
    appId: 'REPLACE_WITH_YOUR_APP_ID',
    messagingSenderId: 'REPLACE_WITH_YOUR_SENDER_ID',
    projectId: 'REPLACE_WITH_YOUR_PROJECT_ID',
    authDomain: 'REPLACE_WITH_YOUR_PROJECT_ID.firebaseapp.com',
    storageBucket: 'REPLACE_WITH_YOUR_PROJECT_ID.appspot.com',
  );

  static const FirebaseOptions android = FirebaseOptions(
    apiKey: 'REPLACE_WITH_YOUR_API_KEY',
    appId: 'REPLACE_WITH_YOUR_APP_ID',
    messagingSenderId: 'REPLACE_WITH_YOUR_SENDER_ID',
    projectId: 'REPLACE_WITH_YOUR_PROJECT_ID',
    storageBucket: 'REPLACE_WITH_YOUR_PROJECT_ID.appspot.com',
  );

  static const FirebaseOptions ios = FirebaseOptions(
    apiKey: 'REPLACE_WITH_YOUR_API_KEY',
    appId: 'REPLACE_WITH_YOUR_APP_ID',
    messagingSenderId: 'REPLACE_WITH_YOUR_SENDER_ID',
    projectId: 'REPLACE_WITH_YOUR_PROJECT_ID',
    storageBucket: 'REPLACE_WITH_YOUR_PROJECT_ID.appspot.com',
    iosBundleId: 'com.urk23cs5015.lab7.lab7FirestoreRecords',
  );
}
```

lib/main.dart

```
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';

import 'firebase_options.dart';
import 'screens/people_records_screen.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  final options = DefaultFirebaseOptions.currentPlatform;
  bool firebaseReady = !options.apiKey.startsWith('REPLACE_WITH');
  if (firebaseReady) {
    try {
```

```
      await Firebase.initializeApp(options: options);
    } catch (_) {
      firebaseReady = false;
    }
  }

  runApp(PeopleRecordsApp(firebaseReady: firebaseReady));
}

class PeopleRecordsApp extends StatelessWidget {
  final bool firebaseReady;

  const PeopleRecordsApp({super.key, required this.firebaseReady});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Employee / Student Records (Firestore)',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),
        useMaterial3: true,
      ),
      home: PeopleRecordsScreen(firebaseReady: firebaseReady),
    );
  }
}
```

Output Screenshots

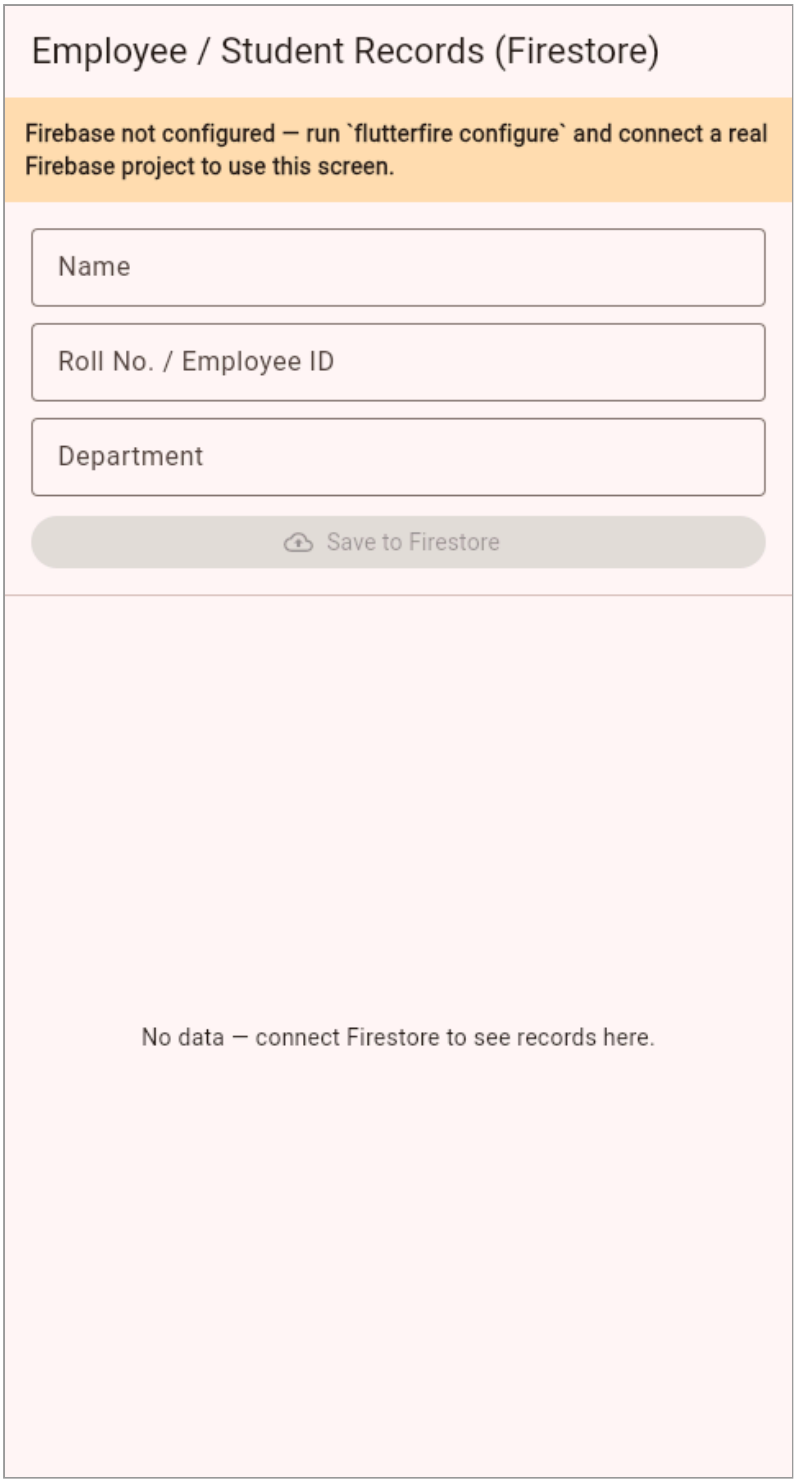


Fig 1. UI shell running against the placeholder Firebase config: the orange banner and disabled Save button confirm the app detects the missing project and degrades gracefully instead of crashing. Once `flutterfire configure` is run with a real project, this banner disappears and the form persists data live to Firestore.

Result

The Flutter application's structure, Create (add) and Read (live StreamBuilder) logic for employee/student records in Google Cloud Firestore were successfully implemented. As no live Firebase project credentials were available in this environment, the app was verified to detect the unconfigured state and degrade gracefully (disabled save, explanatory banner, no data message) rather

than crash, as shown above. Connecting a real Firebase project via `flutterfire configure` activates persistence with no further code changes required.